Module 1: Introduction to Git

- **What is Git?** It is a version control system designed to track changes in source code over time. It essentially acts as a safety net, allowing developers to roll back to previous versions of a project if something goes wrong.
- **Why do we use it?** It is essential for teamwork, as it allows multiple developers to collaborate on the same codebase simultaneously without overwriting each other's work. It maintains a complete history of the project, serving as a reliable backup and recovery system.
- **Git vs. GitHub vs. GitLab:**
    - **Git:** The actual tool installed locally on a computer to manage version history.
    - **GitHub / GitLab:dono same h Cloud-based platforms used to host these repositories online so teams can share code and collaborate easily.**

Module 2: Installation and Setup

- **Installation:** Downloaded and set up the official Git package on the local system.
- **Initial Configuration:** Verified the installation and set up global user identity details using the following commands:
    - `git --version` (To check the installed version)
    - `git config --global user.name "Your Name"` (To set the commit author name)
    - `git config --global user.email "your@email.com"` (To connect the developer email)

Module 3: Basic Git Commands

- **Initializing a Repository (`git init`):** Used to start a new, empty Git repository inside a project folder.
- **Checking Status (`git status`):** Shows the current state of the working directory (which files are modified, untracked, or staged).
- **Staging Files (`git add .`):** Prepares all modified and new files to be included in the next snapshot.
- **Committing Changes (`git commit -m "message"`):** Permanently saves the staged snapshot to the project history with a descriptive message.

- **Viewing History (`git log` / `git log --oneline`):** Displays the chronological list of all previous commits to track the project's evolution.
- **Viewing Changes (`git diff`):** Shows the exact line-by-line differences between the current code and the last saved version.

Module 4: Branching Mechanics

- **What is a Branch?** A branch is an isolated environment within the repository. It allows a developer to build new features or test ideas without disturbing the stable, main codebase.
- **Creating a Branch (`git branch feature-name`):** Sets up a new independent line of development.
- **Switching Branches (`git checkout` / `git switch`):** Moves the working environment from the current branch to another one.
- **Creating & Switching Together (`git checkout -b feature-name`):** A shortcut command that creates a new branch and immediately switches to it.
- **Merging Branches (`git merge feature-name`):** Integrates the completed features and changes from a side branch back into the main branch.
- **Deleting Branches (`git branch -d feature-name`):** Cleans up the repository by removing a branch after its changes have been successfully merged.

Practical Step-by-Step Git Workflow

Step 1: System Setup & Verification

Before starting the project, we check the Git version and configure our global developer identity.

Bash
```
# Check if Git is installed correctly
git --version

# Set up global username and email
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Step 2: Initialize the Project

Create a new project folder (or navigate into an existing one) and initialize it as a Git repository.

Bash
```
# Initialize an empty local Git repository
git init
```

Step 3: Track and Save Changes (The Core Lifecycle)

Create or edit files, check their status, stage them, and save them to the project history.

Bash
```
# Check which files are modified or untracked
git status

# Stage all files/changes to prepare them for a snapshot
git add .
```

```
# Commit and save the staged changes with a descriptive milestone
message
git commit -m "Initial commit"
```

Step 4: View Project Progress and Differences

Check the logs to see the commit history and inspect specific modifications.

Bash

```
# View the full commit history
git log

# View a clean, compressed one-line version of the history
git log --oneline

# View the exact line-by-line changes made before staging
git diff
```

Step 5: Feature Branching (Isolating Work)

Create a new branch to work on a feature safely without messing up the main production code.

Bash

```
# Create a new branch named 'feature-login'
git branch feature-login

# Switch over to the newly created branch
git checkout feature-login
# (Alternative modern command: git switch feature-login)

# SHORTCUT: Create and switch to a new branch in one single command
git checkout -b feature-login
```

Step 6: Merging and Cleaning Up

Once the feature is tested and complete, switch back to the main branch, pull the changes in, and clean up the repository.

Bash

```
# Switch back to the main branch first
git checkout main

# Merge the changes from 'feature-login' into the main branch
git merge feature-login

# Delete the feature branch now that it's safely merged
git branch -d feature-login
```